

인공지능수학 기말프로젝트 보고서

선형회귀 · 다중 클래스 로지스틱 회귀 · 딥러닝

학번: _____

이름: _____

제출일: 2026. 6. 19.

프로젝트 개요

이번 기말프로젝트에서는 수업에서 배운 세 가지 모델, 즉 선형회귀, 다중 클래스 로지스틱 회귀(소프트맥스 회귀), 딥러닝(임베딩과 다층 퍼셉트론)을 각각 서로 다른 데이터셋에 적용해 보았다. 세 데이터셋 모두 수업에서 다룬 것(붓꽃, MNIST, Fashion MNIST)이나 교재의 단골 예제와 겹치지 않게 골랐고, 라이브러리에 결과를 맡기기만 하는 것이 아니라 수업에서 유도한 수식(정규방정식, 경사하강법, 소프트맥스와 크로스 엔트로피의 기울기)을 NumPy로 직접 구현한 뒤 scikit-learn의 결과와 비교하는 방식으로 구현이 올바른지 검증하였다.

구분	데이터셋	표본 수	과제 유형	핵심 결과
프로젝트 1	Diamonds (ggplot2)	53,917	회귀 (가격 예측)	R^2 0.912 $\rightarrow$ 0.952 (로그변환)
프로젝트 2	Wine (UCI)	178	3-클래스 분류	정확도 97.2% (직접 구현 = sklearn)
프로젝트 3	InstEval (ETH Zurich)	73,421	5-클래스 분류 (평점 예측)	30.7% (다수결 기준선 24.0%)

실험 환경

Python 3.10, NumPy 2.2, scikit-learn 1.7, TensorFlow(CPU) 2.21, matplotlib 3.10 환경에서 진행하였다. 모든 실험은 난수 시드를 42로 고정하여 재현 가능하도록 하였고, 코드는 별도의 코드 PDF로 제출한다.

생성형 AI 활용 표기: 본 프로젝트는 Anthropic의 Claude를 활용하여 진행하였다. 구체적으로 (1) 세 프로젝트의 파이썬 코드 작성과 실험 실행, (2) 실험 결과 수치·그림 생성, (3) 보고서 초안의 구성과 이론 설명 정리에 Claude를 사용하였고, 이후 내용을 직접 검토하고 이해하여 보고서를 완성하였다. 데이터셋 선택 기준, 실험 설계 방향, 최종 검수는 본인이 수행하였다.

프로젝트 1. 선형회귀 — 다이아몬드 가격 예측 (Diamonds)

1-1. 데이터셋 선정과 분석, 문제 정의

Diamonds 데이터셋은 R의 ggplot2 패키지에 포함된 공개 데이터로, 53,940개의 다이아몬드에 대해 가격(price, 326~18,823 USD)과 9개의 특성을 기록하고 있다. 파이썬에서는 pydataset 패키지로 불러왔다. 특성은 무게(carat), 컷 등급(cut: Fair~Ideal 5단계), 색 등급(color: J~D 7단계), 투명도 등급(clarity: I1~IF 8단계), 깊이 비율(depth), 테이블 비율(table), 그리고 세 방향 치수 x, y, z(mm)이다.

이 데이터셋을 고른 이유는 세 가지이다. 첫째, 5만 개가 넘는 표본으로 수업 예제보다 규모가 훨씬 크다. 둘째, 순서가 있는 범주형 변수(cut, color, clarity)가 섞여 있어 인코딩이라는 전처리 문제를 직접 다뤄야 한다. 셋째, 탐색 결과 가격 분포가 오른쪽으로 길게 치우쳐 있고 carat과 price의 관계가 곡선 형태라서(그림 1), 단순 선형 모형이 어디까지 통하는지, 무엇으로 보완할 수 있는지를 분석하기에 좋은 대상이기 때문이다.

문제 정의: 다이아몬드의 물리적 특성 9개로부터 가격을 예측하는 회귀 문제. 접근 방안: MSE를 손실함수로 하는 선형회귀를 (1) 정규방정식, (2) 배치 경사하강법으로 직접 구현하고 (3) scikit-learn과 비교해 구현을 검증한 뒤, 로그 변환으로 성능을 개선한다.

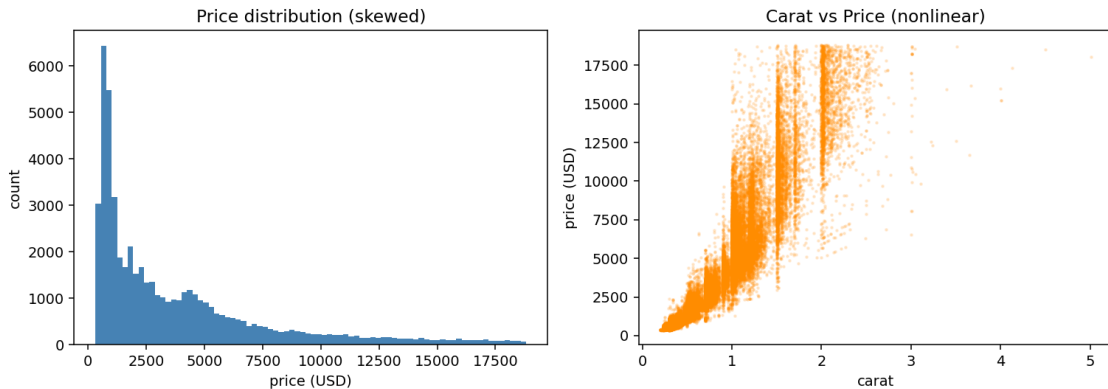


그림 1. 가격 분포(왼쪽, 오른쪽으로 치우침)와 carat-price 산점도(오른쪽, 비선형 관계)

1-2. 구현

전처리에서는 먼저 x, y, z 중 하나라도 0인 행(물리적으로 불가능한 기록 오류) 20개와, y=58.9mm·z=31.8mm처럼 기록 오류로 보이는 극단값 3개를 제거하여 53,917행을 사용했다. cut/color/clarity는 등급에 명확한 순서가 있어서 원-핫 인코딩 대신 품질이 좋을수록 큰 정수가 되도록 순서형 인코딩을 적용했다. 데이터는 8:2로 학습/테스트를 나누고, 학습셋의 평균과 표준편차만으로 두 셋을 모두 표준화하여 테스트 정보가 학습에 새어 들어가지 않게 했다.

모델은 세 가지 방법으로 학습했다. 첫째, 정규방정식 $w^* = (X^T X)^{-1} X^T y$ 를 NumPy 행렬 연산으로 직접 계산했다. 둘째, 배치 경사하강법으로 $\nabla L = (2/N) X^T (Xw - y)$ 를 매 epoch 계산해 w를 갱신했다(학습률 0.1, 300 epoch). 셋째, scikit-learn의 LinearRegression으로 같은 데이터를 학습했다. 정규방정식과 scikit-learn의 가중치가 소수점 셋째 자리까지 완전히 일치하여 구현이 올바름을 확인했다.

1-3. 결과

방법	테스트 RMSE (USD)	테스트 MAE (USD)	테스트 R ²
정규방정식	1,174.45	786.93	0.9125
경사하강법 (lr=0.1, 300ep)	1,191.80	816.15	0.9099
scikit-learn	1,174.45	786.93	0.9125
로그변환 + 정규방정식	866.88	455.39	0.9523

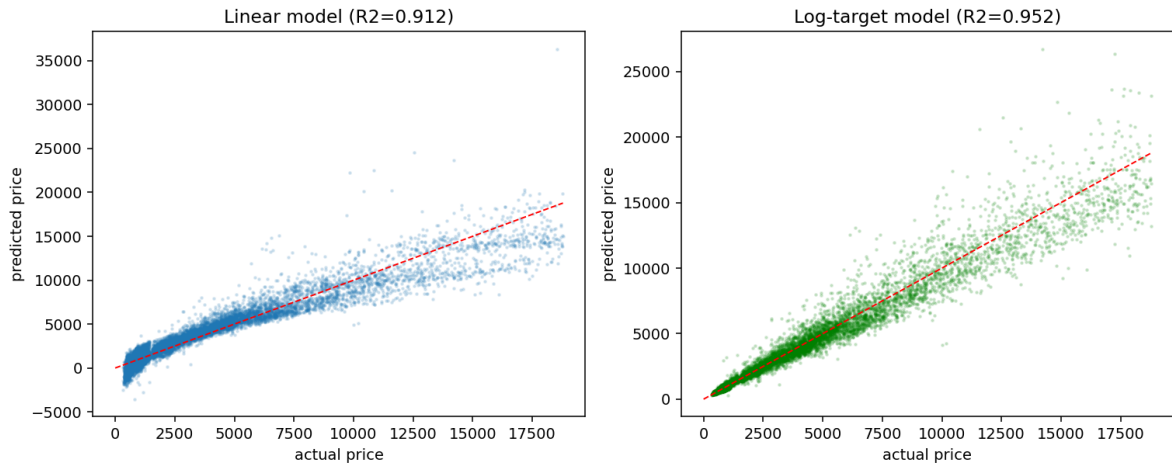


그림 2. 예측 vs 실제 가격. 기본 선형 모형(왼쪽)은 고가 구간에서 과소예측과 휘어짐이 보이고, 로그변환 모형(오른쪽)은 전 구간에서 대각선에 더 가깝다.

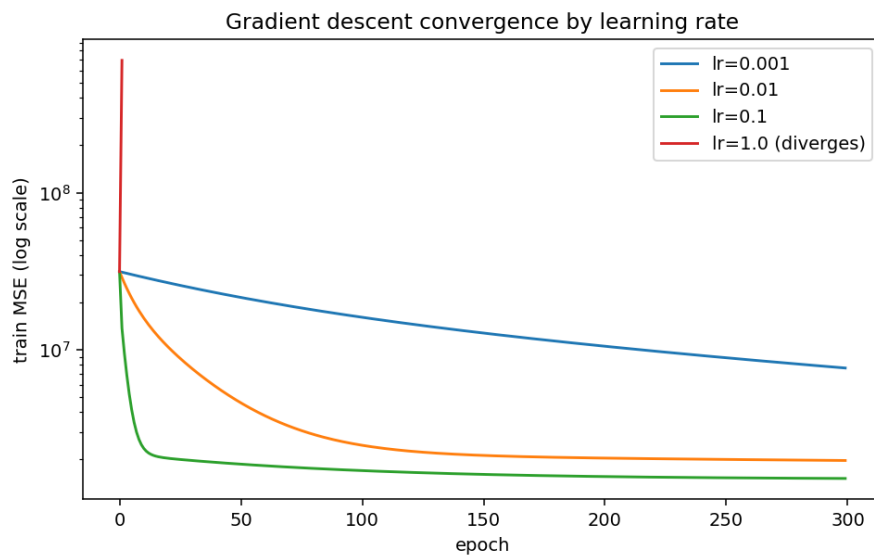


그림 3. 학습률별 경사하강법 수렴. 0.001은 너무 느리고, 0.1은 빠르게 수렴하며, 1.0은 손실이 폭주(발산)한다.

1-4. 이론적 설명

모형과 손실함수. 선형회귀는 입력 특성의 선형 결합으로 출력을 예측하는 모형이다.

$$y_p = w_0 + w_1x_1 + w_2x_2 + \dots + w_9x_9$$

학습은 예측과 정답의 차이를 제공해 평균한 MSE를 최소화하는 가중치를 찾는 일이다.

$$L(w) = (1/N) \sum_i (y_i - y_{p,i})^2$$

정규방정식. MSE는 w 에 대한 2차식(아래로 볼록)이므로 기울기가 0이 되는 점이 유일한 최솟값이다. 편향 항을 포함한 설계행렬 X 에 대해 $\partial L / \partial w = 0$ 을 정리하면 $X^T X w = X^T y$, 즉

$$w^* = (X^T X)^{-1} X^T y$$

를 얻는다. 한 번의 행렬 계산으로 정확한 최적해를 얻을 수 있는 대신, 특성 수가 많아지면 역행렬 계산 비용이 커지고 $X^T X$ 가 특이행렬에 가까우면 수치적으로 불안정해진다.

경사하강법. 손실의 기울기 $\nabla L = (2/N) X^T (Xw - y)$ 는 손실이 가장 빠르게 증가하는 방향이므로, 그 반대 방향으로 학습률 α 만큼 이동하는 것을 반복한다: $w \leftarrow w - \alpha \nabla L$. 그림 3이 보여주듯 α 가 너무 작으면 수렴이 느리고, 적당하면 빠르게 수렴하며, 너무 크면 최솟값을 지나쳐 반대편으로 더 멀리 튕겨 나가는 일이 반복되어 발산한다. 실험에서 $\alpha=1.0$ 일 때 손실이 10^9 를 넘어 폭주한 것이 정확히 이 경우이다.

표준화의 역할. carat(0.2~5)과 z(2~32mm)처럼 특성의 스케일이 다르면 손실함수의 등고선이 길쭉한 타원이 되어, 경사하강법이 좁은 골짜기를 지그재그로 내려가느라 수렴이 느려진다. 각 특성을 평균 0, 표준편차 1로 맞추면 등고선이 원에 가까워져 같은 학습률로도 모든 방향에서 고르게 수렴한다.

R^2 의 의미. $R^2 = 1 - \text{SSE}/\text{SST}$ 로, 평균만으로 예측했을 때의 오차(SST) 중 모형이 줄인 비율이다. 테스트 R^2 가 0.9125라는 것은 가격 분산의 약 91%를 9개 특성의 선형 결합으로 설명했다는 뜻이다.

로그 변환이 효과적인 이유. 그림 1에서 가격은 오른쪽으로 길게 치우쳐 있고, 그림 4의 잔차는 가격이 커질수록 퍼짐이 커지는 이분산성을 보인다. 가격이 '특성 효과의 곱'에 가깝게 움직인다고 보면 $\log(\text{price})$ 는 특성들의 합, 즉 선형 모형에 잘 맞는 형태가 된다. 실제로 $\log(\text{price})$ 를 목표로 학습하자 로그 공간 R^2 가 0.979, 달러로 환원한 뒤에도 R^2 0.952, MAE는 787달러에서 455달러로 줄었다.

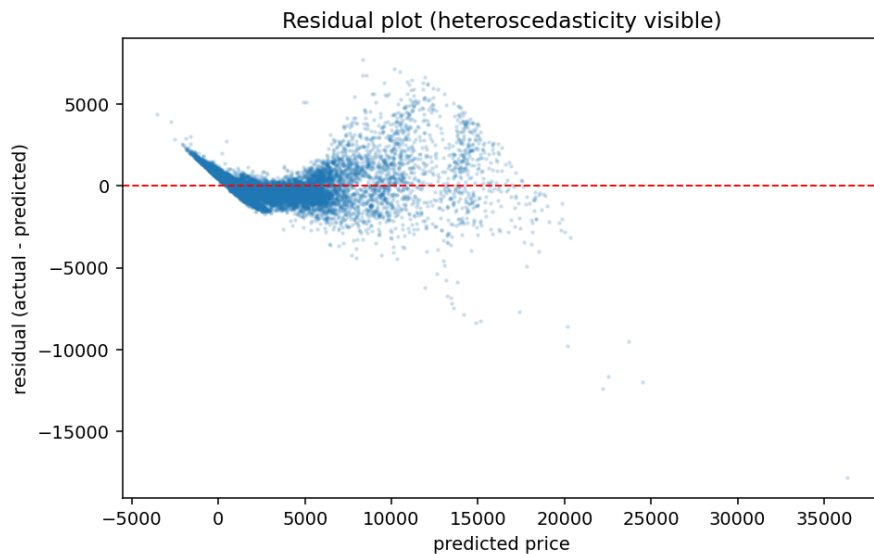


그림 4. 기본 선형 모형의 잔차. 0 주위에 고르게 퍼지지 않고 곡선 패턴과 이분산성이 보인다 — 선형성·등분산 가정이 깨져 있다는 신호.

1-5. 문제점과 해결방안

문제 1 — 로그 모형의 예측 폭주. 처음 로그변환 모형을 만들었을 때 테스트 MSE가 10^{19} 수준으로 터지는 문제를 겪었다. 원인을 추적해 보니 $y=58.9\text{mm}$ 짜리 기록 오류 행이 테스트셋에 들어 있었고, 표준화 후 이 행의 특성값이 극단값이 되어 로그 공간 예측이 커졌으며, exp로 환원하는 순간 천문학적 숫자가 된 것이었다. 학습 자체가 아니라 이상치 하나가 평가 지표 전체를 무너뜨린 사례로, 치수 20mm 이상인 행을 제거하여 해결했다. 제품 손실과 exp 변환은 둘 다 극단값에 민감하므로 회귀에서는 이상치 점검이 선행되어야 한다는 점을 직접 확인했다.

문제 2 — 경사하강법이 최적해에 못 미침. 300 epoch 후에도 경사하강법의 학습 MSE(약 1,515,000)는 정규방정식이 도달한 최적값(약 1,482,000)에 못 미쳤고 테스트 R^2 도 0.910으로 약간 낮았다. 표준화를 했어도 특성 간 상관(아래 문제 3)으로 등고선이 완전한 원이 아니어서 일부 방향의 수렴이 느린 것이 원인이다. epoch를 늘리거나, 학습률을 점차 줄이는 스케줄, 모멘텀·Adam 같은 적응적 방법을 쓰면 더 가까워질 수 있다.

문제 3 — 다중공선성. 학습된 가중치를 보면 x에 -2,129, y에 +2,614, z에 -1,528로 부호가 들쭉날쭉한데, 치수가 클수록 비싸다는 직관과 어긋난다. 이는 carat이 부피($x \cdot y \cdot z$)에 거의 비례해서 네 특성이 강하게 상관되어 있기 때문이다. 상관이 강하면 $X^T X$ 가 특이행렬에 가까워져 개별 가중치가 불안정해지고 해석이 어려워진다. 예측 성능 자체는 유지되지만, 릿지 규제($L(w)$ 에 $\lambda \|w\|^2$ 추가)로 가중치를 안정화하거나 carat만 남기는 특성 선택이 해결책이 된다.

문제 4 — 남은 비선형성. 로그변환 후에도 R^2 0.952에서 남는 오차는, 잔차의 곡선 패턴이 보여주듯 선형 모형으로는 답을 수 없는 비선형 관계(특성 간 상호작용 포함) 때문이다. 다항 특성을 추가하거나 트리 계열 모델을 쓰면 더 줄일 수 있겠지만, 이번 프로젝트는 수업 범위인 선형 모형의 한계를 확인하는 데 의미를 두었다.

프로젝트 2. 다중 클래스 로지스틱 회귀 — 와인 품종 분류 (Wine)

2-1. 데이터셋 선정과 분석, 문제 정의

UCI Wine 데이터셋은 이탈리아 한 지역에서 재배된 세 품종(cultivar)의 와인 178개를 화학적으로 분석한 자료로, scikit-learn의 load_wine으로 불러왔다. 특성은 알코올 도수, 말산, 회분, 플라보노이드, 색 강도, proline 등 13개이고, 클래스 분포는 59/71/48로 약간의 불균형이 있다. 수업의 Iris(특성 4개)와 같은 3-클래스 문제이지만 특성이 13개로 많고, 특히 특성 간 스케일 차이가 극단적이다: proline의 표준편차는 약 314인데 nonflavanoid_phenols는 약 0.12로 2,600배가 넘는다. 이 성질 때문에 수업에서 배운 '표준화가 경사하강법에 왜 필요한가'를 실험으로 확인하기에 적합한 데이터라고 판단했다. Wine은 과제 안내문에 예시로 제시된 데이터셋 중 하나이며, 수업 시간에 다룬 데이터셋은 아니다.

문제 정의: 13개 화학 성분 측정값으로 와인 품종(3 클래스)을 분류한다. 접근 방안: 소프트맥스 회귀를 수업에서 유도한 기울기 식으로 직접 구현해 학습하고, scikit-learn의 다중 클래스 LogisticRegression과 정확도를 비교해 검증한다. 추가로 표준화 유무와 학습률이 수렴에 미치는 영향을 실험한다.

2-2. 구현

데이터는 클래스 비율을 유지하며(stratify) 8:2로 나누고 학습셋 통계량으로 표준화했다. 직접 구현한 학습 루프는 다음 순서로 동작한다. (1) 정답을 원-핫 행렬 Y로 변환, (2) 선형 점수 $U = XW + b$ 계산 (W는 13×3 가중치 행렬), (3) 소프트맥스로 확률 P 계산 — 이때 행마다 최댓값을 빼서 exp의 오버플로를 막았다, (4) 크로스 엔트로피 손실 기록, (5) 기울기 $\partial L / \partial W = X^T(P - Y) / N$ 로 W와 b를 갱신(학습률 0.5, 500 epoch). 평가는 혼동행렬과 클래스별 정밀도-재현율로 했고, 13차원 전체는 그릴 수 없으므로 판별력이 좋은 두 특성(alcohol, flavanoids)만으로 보조 모델을 학습해 수업 자료와 같은 방식의 2차원 결정경계 그림을 그렸다.

2-3. 결과

설정	테스트 정확도
직접 구현 (표준화, lr=0.5, 500ep)	97.22% (36개 중 35개)
scikit-learn LogisticRegression	97.22%
표준화 없음 (lr=0.5)	69.44%
표준화 없음 + 아주 작은 학습률 (1e-6)	52.78%
특성 2개(alcohol, flavanoids)만 사용	91.67%

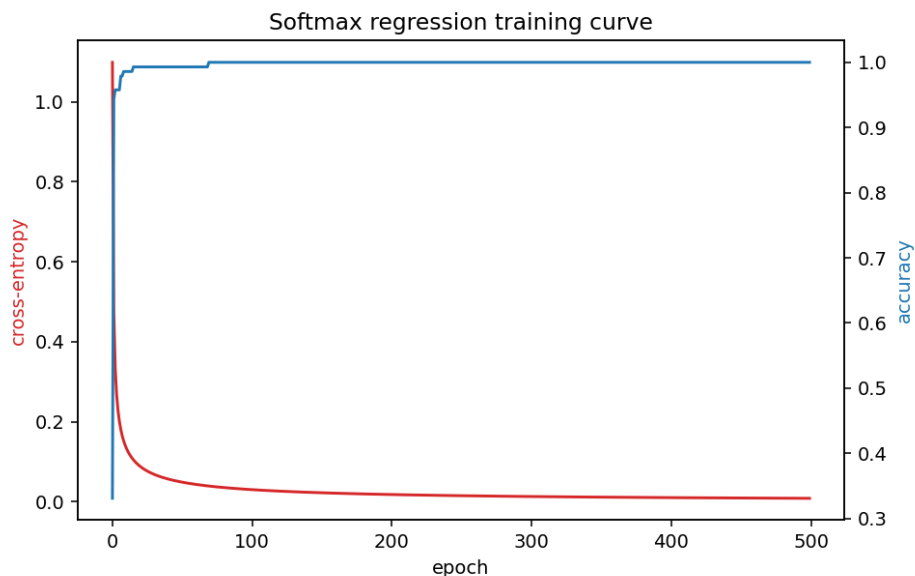


그림 5. 직접 구현 모델의 학습 곡선. 크로스 엔트로피(빨강)가 단조 감소하고 학습 정확도(파랑)는 100%에 도달한다.

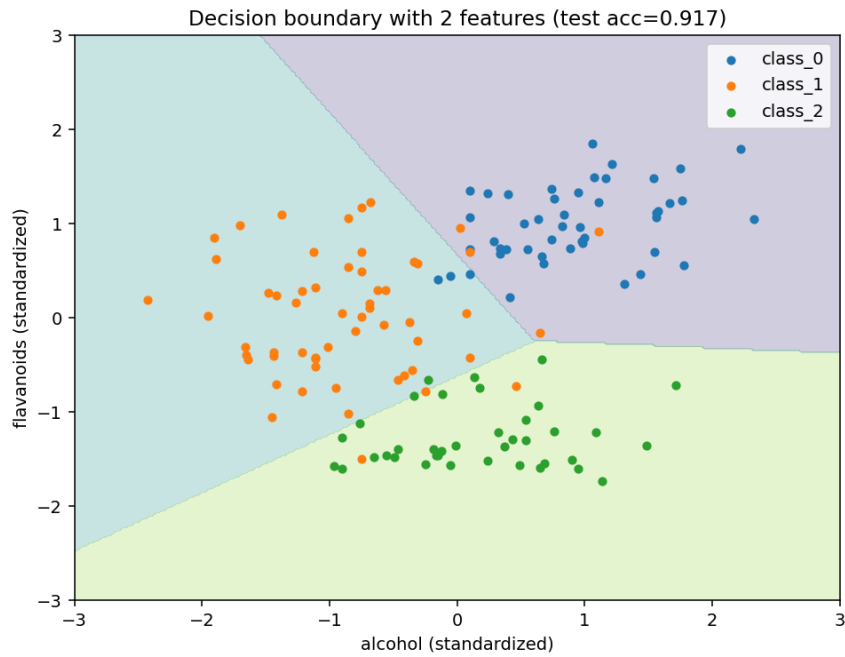


그림 6. 두 특성만 사용한 소프트맥스 회귀의 결정경계. 세 영역의 경계가 직선으로 나뉜다 — 소프트맥스 회귀가 선형 분류기임을 보여준다.

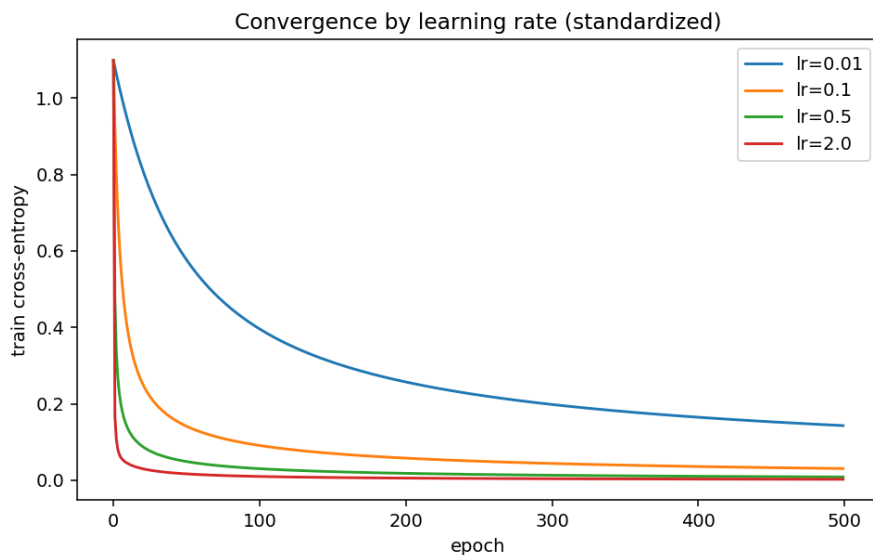


그림 7. 학습률별 크로스 엔트로피 수렴(표준화된 데이터). 0.01은 500 epoch에도 부족하고 0.5 이상은 빠르게 수렴한다.

2-4. 이론적 설명

선형 점수에서 확률로: 소프트맥스. 클래스가 3개이므로 클래스마다 하나씩, 세 개의 선형 점수를 만든다.

$$u_k = w_{0k} + w_{1k}x_1 + \dots + w_{13,k}x_{13} \quad (k = 0, 1, 2)$$

이 점수들은 임의의 실수이므로 그대로는 확률이 아니다. 소프트맥스는 각 점수에 exp를 취해 모두 양수로 만든 뒤 합으로 나누어, 합이 1인 확률 분포로 바꾼다.

$$P(\text{class } k) = e^{u_k} / (e^{u_0} + e^{u_1} + e^{u_2})$$

exp는 단조 증가라 점수의 순위를 보존하면서, 점수 차이를 지수적으로 증폭해 가장 큰 점수에 확률을 몰아준다. 이진 분류의 시그모이드는 클래스가 2개일 때 소프트맥스의 특수한 경우와 같다.

원-핫 인코딩과 크로스 엔트로피. 정답 클래스 1을 [0,1,0]처럼 정답 자리만 1인 벡터로 표현하면, 크로스 엔트로피 손실은 다음과 같이 정의된다.

$$L = -(1/N) \sum_i \sum_k y_{ik} \log p_{ik}$$

원-핫 벡터 때문에 안쪽 합에서는 정답 클래스 항만 남으므로, 이 손실은 결국 '모델이 정답에 부여한 확률의 -log 평균'이다. 정답 확률이 1이면 손실 0, 정답 확률이 0에 가까우면 손실이 무한대로 커진다. 수업에서 다른 정보이론의 관점으로 말하면 $-\log p$ 는 '놀라움(정보량)'이고, 크로스 엔트로피 최소화는 정답을 보고 덜 놀라는 모델을 만드는 일이다. MSE와 달리 잘못된 확신에 매우 큰 벌점을 주기 때문에 분류에 적합하다.

기울기가 $P - Y$ 로 단순해지는 이유. 소프트맥스와 크로스 엔트로피를 연쇄법칙(chain rule)으로 함께 미분하면 복잡한 중간 항들이 약분되어 $\partial L / \partial u = P - Y$ 라는 깔끔한 결과가 남는다. 여기에 $u = XW + b$ 의 미분을 한 번 더 연결하면

$$\partial L / \partial W = X^T (P - Y) / N$$

이 된다. '예측 확률에서 정답을 뺀 것'이 그대로 오차 신호가 되는 구조라서, 구현도 행렬 곱 두 번으로 끝난다. 수업에서 유도한 이 식을 코드의 갱신식으로 그대로 옮겼고, scikit-learn과 정확도가 일치한 것이 유도와 구현이 맞다는 증거가 된다.

결정경계의 기하학. 두 클래스 i, j 의 경계는 $u_i = u_j$ 인 점들, 즉 가중치 차이 벡터를 법선으로 하는 직선(초평면)이다. 그림 6에서 세 영역이 직선으로 나뉘는 이유이며, 클래스 내부로 들어갈수록 점수 차가 커져 확률이 1에 가까워진다.

2-5. 문제점과 해결방안

문제 1 — 표준화 없이는 학습이 사실상 실패. 같은 코드에서 표준화만 빼면 정확도가 97.2%에서 69.4%로 떨어진다. 원인은 스케일이 압도적으로 큰 proline(표준편차 약 314)이 기울기 $X^T(P-Y)$ 를 지배해, 손실 등고선이 극단적으로 길쭉해지기 때문이다. 학습률 0.5는 proline 방향에서는 너무 커서 진동하고, 다른 특성 방향에서는 거의 움직이지 못한다. 그렇다고 학습률을 proline에 맞춰 $1e-6$ 으로 줄이면 발산은 없지만 500 epoch 동안 다른 방향이 거의 학습되지 않아 52.8%에 그쳤다. 단일 학습률로는 해결이 안 되는 구조적 문제이고, 표준화로 모든 특성의 스케일을 맞추는 것이 올바른 해결책을 실험으로 확인했다.

문제 2 — 작은 데이터와 평가의 불안정성. 전체가 178개라 테스트셋이 36개뿐이고, 한 개만 틀려도 정확도가 2.8%p 움직인다. 지금의 97.22%도 분할 운에 따라 달라질 수 있는 수치이므로, 더 신뢰성 있는 평가를 위해서는 k -겹 교차검증으로 여러 분할의 평균을 보고하는 것이 바람직하다.

문제 3 — 학습 정확도 100%와 과적합 가능성. 학습 손실이 0.0086까지 내려가고 학습 정확도가 100%인 것은 모델이 학습셋을 완전히 외웠다는 뜻이기도 하다. 이번에는 테스트 정확도가 97.2%로 높아 일반화가 잘 되었지만, 특성 수가 표본 수에 비해 많은 상황에서는 가중치가 비대해지며 과적합이 생길 수 있다. 손실에 $\lambda \|W\|^2$ 항을 더하는 L2 규제(실제로 scikit-learn의 기본 설정이 이 방식이다)가 표준적인 대비책이다.

문제 4 — 남은 오분류 1건. 혼동행렬을 보면 class 2 하나가 class 1로 분류되었다. 두 특성 그림(그림 6)에서도 class 1과 2의 경계 부근에 겹치는 표본들이 보이는데, 선형 결정경계로는 분리할 수 없는 영역이다. 특성을 모두 쓰면 13차원에서는 더 잘 분리되지만 완전하지는 않으며, 비선형 경계가 필요하다면 커널 방법이나 신경망(프로젝트 3)이 다음 단계가 된다.

프로젝트 3. 딥러닝 — 강의평가 점수 예측 (InstEval)

3-1. 데이터셋 선정과 분석, 문제 정의

InstEval은 ETH Zurich(스위스 취리히 연방공대)의 실제 강의평가 기록 73,421건을 담은 공개 데이터셋으로, R의 lme4 패키지에 포함되어 있으며 파이썬에서는 pydataset으로 불러왔다. 한 행은 '학생 s가 교수 d의 강의에 평점 y(1~5)를 주었다'는 기록이고, 학생 ID(2,972명), 교수 ID(1,128명) 외에 학생 연차(studage), 수강 후 경과 학기(lectage), 필수과목 여부(service), 학과(dept)가 들어 있다. 평점 분포는 1점부터 10,186 / 12,951 / 17,609 / 16,921 / 15,754개로(그림 8), 최빈 클래스(3점)만 찍는 다수결 기준선이 약 24.0%이다.

이 데이터셋을 고른 이유는 난이도의 성격이 앞의 두 프로젝트와 완전히 다르기 때문이다. 첫째, 입력의 핵심이 '누가(2,972개 ID), 누구를(1,128개 ID)'이라는 합계 4,100개짜리 희소 범주형 변수다. 이런 입력은 원-핫 인코딩이 비현실적이어서(4,100차원), 딥러닝 고유의 기법인 임베딩(embedding) 없이는 다루기 어렵다. 둘째, 사람의 평가라는 본질적으로 시끄러운 신호를 예측하는 문제라, 같은 교수의 같은 강의에도 학생마다 점수가 갈린다. 이미지 분류처럼 정답이 명확한 과제와 달리 도달 가능한 정확도 자체가 낮고, '기준선 대비 얼마나 더 맞는가'와 '틀릴 때 얼마나 가깝게 틀리는가'를 분석하는 것이 과제의 중심이 된다. 셋째, 학생 입장에서 강의평가라는 소재가 직관적으로 와닿고, 교재에서 흔히 쓰는 데이터(붓꽃, 손글씨 등)와 전혀 겹치지 않는다.

문제 정의: (학생 ID, 교수 ID, 연차, 수강 시점, 필수 여부, 학과)로부터 평점(1~5)을 예측하는 5-클래스 분류. 접근 방안: 학생/교수 ID를 임베딩으로 변환해 MLP에 연결하는 모델을 중심으로, 기준선 2개(다수결, ID 없이 특성만 쓴 소프트맥스 회귀)와 비교하고, 조기 종료·망 깊이·활성화 함수 실험과 순서형(ordinal) 지표 분석을 더한다.

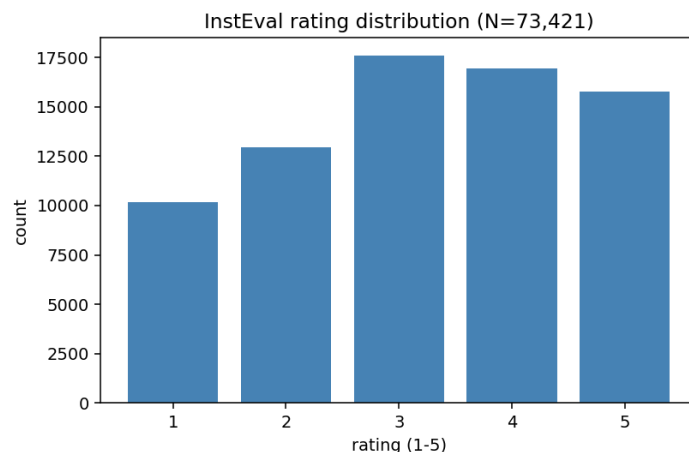


그림 8. 평점 분포. 3점이 최빈(24.0%)이고 클래스 불균형은 심하지 않다.

3-2. 구현

전처리: 평점 1~5를 0~4로 바꾸고(케라스의 정수 레이블 크로스 엔트로피 입력 형식), 학생/교수 ID는 중간에 빈 번호가 있어 0부터 시작하는 연속 정수로 다시 매겼다(임베딩층은 ID를 행렬의 행 번호로 쓰기 때문). 카디널리티가 작은 나머지 4개 변수는 원-핫 인코딩해 26차원이 되었다. 데이터는 학습 64% / 검증 16% / 테스트 20%로 나눴다 (46,988 / 11,748 / 14,685건, 클래스 비율 유지).

모델 구성(공통: Adam, 크로스 엔트로피, 배치 512): A. ID를 아예 쓰지 않고 원-핫 특성 26개만 받는 소프트맥스 회귀 — 'ID 정보가 없으면 어디까지 가능한가'의 하한선. B. 학생·교수 ID를 각각 16차원 임베딩으로 바꾸고 원-핫 26차원과 이어 붙여(58차원) 은닉층 64(ReLU)와 드롭아웃 0.3을 거쳐 소프트맥스로 출력하는 주 모델, 고정 12 epoch. B'. B와 같은 구조에 조기 종료(검증 손실이 3 epoch 연속 개선되지 않으면 정지하고 최적 시점 가중치로 복원)를 적용. C/D. 임베딩 뒤 은닉층을 64노드 6개로 깊게 쌓고 활성화 함수만 ReLU/시그모이드로 다르게 한 비교쌍(고정 10 epoch).

3-3. 결과

모델	설정	테스트 정확도
다수결 기준선	무조건 3점 예측	23.98%
A. 특성만 소프트맥스	ID 미사용, 원-핫 26차원	24.07%
B. 임베딩+MLP	고정 12 epoch	28.32%
B'. 임베딩+MLP+조기종료	epoch 6에서 정지, 최적 가중치 복원	30.66%
C. 깊은 망 (ReLU)	은닉 64×6층, 고정 10 epoch	28.61%
D. 깊은 망 (시그모이드)	은닉 64×6층, 고정 10 epoch	30.02%

최종 모델 B'의 순서형 지표: 평균절대오차(MAE) 1.112점, 확률 가중 기대평점으로 예측하면 MAE 1.013점, ± 1 점까지 허용한 정확도 68.2%.

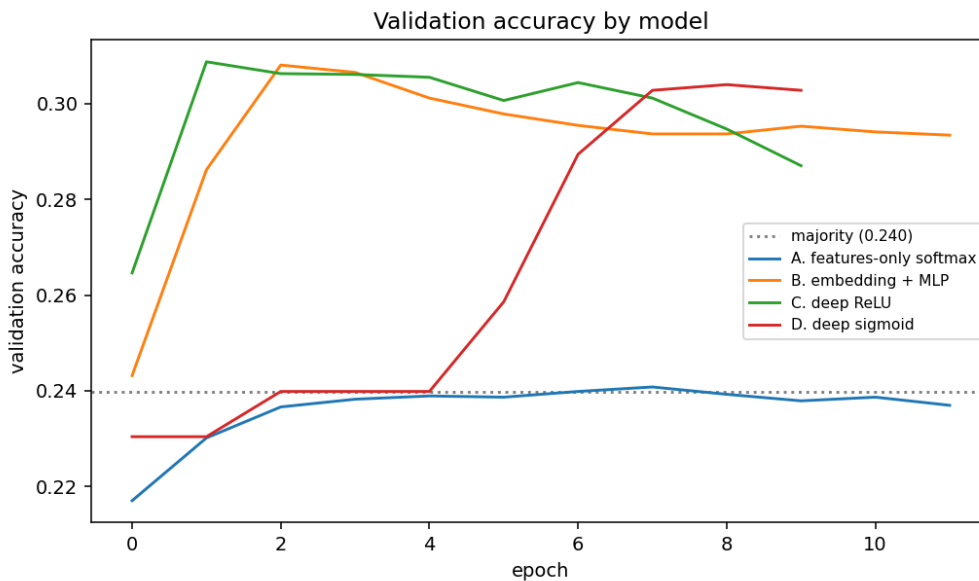


그림 9. 모델별 검증 정확도. ID를 쓰지 않는 A(파랑)는 다수결 수준(점선)을 못 벗어난다. B(주황)와 C(초록)는 1~2 epoch 만에 정점을 찍고 내려오며(과적합), 시그모이드 D(빨강)는 5 epoch 동안 기준선에 붙어 정체하다가 뒤늦게 상승한다.

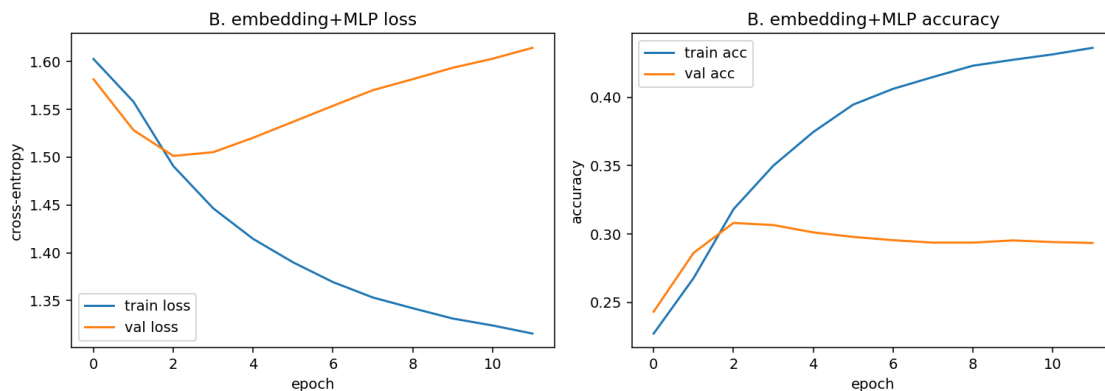


그림 10. 주 모델 B(고정 12 epoch)의 학습/검증 곡선. 검증 손실은 2~3 epoch에서 최저점을 찍고 다시 상승하는데(왼쪽), 학습 정확도는 43.6%까지 계속 오르고 검증 정확도는 29%대로 내려간다(오른쪽) — 전형적인 과적합.

3-4. 이론적 설명

원-핫 인코딩의 한계와 임베딩. 학생 ID를 원-핫으로 표현하면 2,972차원 벡터에 1이 하나 있는 형태가 된다. 이 벡터에 가중치 행렬 $W(2,972 \times 16)$ 를 곱하면, 행렬 곱의 정의에 따라 결과는 W 에서 해당 학생의 행 하나를 꺼낸 것과 정확히 같다. 임베딩층은 이 사실을 이용해, 거대한 원-핫 벡터를 실제로 만들지 않고 행렬에서 행을 직접 꺼내 쓰는 층이다. 즉 임베딩은 '원-핫 입력에 대한 선형층'과 수학적으로 동일하며, 수업에서 배운 행렬 곱 한 번으로 이해된다. 중요한 것은 이 행렬 W 도 역전파로 함께 학습된다는 점이다. 학습이 진행되면 평가 성향이

비슷한 학생들, 평가받는 패턴이 비슷한 교수들이 16차원 공간에서 가까운 벡터를 갖게 된다 — 모델이 입력의 '표현' 자체를 학습하는 것이고, 이것이 고전적 방법과 구별되는 딥러닝의 핵심 성질이다.

출력층은 프로젝트 2와 같다. 마지막 층은 5개 점수에 대한 소프트맥스이고 손실은 크로스 엔트로피이므로, 출력층만 보면 이 모델은 '58차원 학습된 특징 위의 소프트맥스 회귀'다. 따라서 $\partial L / \partial u = P - Y$ 라는 프로젝트 2의 유도가 출력층에 그대로 적용되고, 그 오차 신호가 연쇄법칙(chain rule)을 따라 은닉층을 지나 임베딩 행렬의 해당 행까지 거꾸로 전달된다. 이것이 역전파이며, Keras의 fit이 자동으로 수행하는 계산이다.

기울기 소실 — 시그모이드 정체의 수학적 원인. 시그모이드의 도함수 $\sigma'(u) = \sigma(u)(1-\sigma(u))$ 는 최대 0.25이다. 역전파는 층마다 활성화 함수의 도함수를 곱하므로, 은닉 6층이면 오차 신호가 최대 $0.25^6 \approx 0.0002$ 배까지 줄어든다. 그림 9에서 시그모이드 망(D)이 5 epoch 동안 다수결 수준에 머문 것이 바로 이 현상이다. 입력에 가까운 층(임베딩 포함)이 거의 갱신되지 못하다가, 출력 쪽 층들이 먼저 자리를 잡으면서 뒤늦게 신호가 통하기 시작한 것이다. ReLU는 양수 구간 도함수가 정확히 1이라 신호가 줄지 않고 전달되어 첫 epoch부터 학습이 진행된다(C).

드롭아웃과 Adam, 미니배치. 주 모델의 드롭아웃 0.3은 학습 중 매 배치마다 은닉 노드 30%를 무작위로 꺼서 특정 노드 조합에 의존한 암기를 방해한다. 최적화는 46,988건을 배치 512로 나눠 epoch당 92번 갱신하는 미니배치 방식이고, Adam은 기울기의 이동평균(모멘텀)과 기울기 제곱의 이동평균으로 파라미터마다 보폭을 자동 조절한다 — 프로젝트 1·2에서 학습률 하나를 수동으로 고르며 겪은 발산/정체 문제를 옵티마이저가 완화해 준다.

조기 종료(early stopping). 그림 10처럼 학습 손실은 계속 내려가도 검증 손실은 어느 시점부터 다시 올라간다. 그 분기점 이후의 학습은 '학습셋 암기'이므로, 검증 손실이 patience(3 epoch) 동안 개선되지 않으면 멈추고 가장 좋았던 시점의 가중치로 되돌리는 것이 조기 종료다. epoch 수라는 하이퍼파라미터를 사람이 찍는 대신 검증 데이터가 고르게 하는 장치로, 이번 실험에서 가장 큰 단일 성능 개선(+2.3%p)을 가져왔다.

순서형 레이블과 기대 평점. 크로스 엔트로피는 평점 1~5를 순서 없는 5개 명목 클래스로 취급해서, 4점을 5점으로 틀리는 것과 1점으로 틀리는 것을 똑같이 벌점한다. 평점의 순서 정보를 살리는 간단한 방법으로, 소프트맥스 확률로 가중평균한 기대 평점 $\sum k \cdot p_k$ 를 예측값으로 쓰면 MAE가 1.112에서 1.013으로 줄었다 — 모델이 가진 불확실성 정보(확률 분포)를 점 예측에 활용한 효과다.

3-5. 문제점과 해결방안

문제 1 — 빠른 과적합. 주 모델 B를 고정 12 epoch 학습하면 학습 정확도는 43.6%까지 오르지만 검증 정확도는 3 epoch 이후 하락해 테스트 28.32%에 그친다(그림 10). 파라미터의 대부분이 임베딩 행렬(약 6.6만 개)인데 학생 1인당 평균 평가 수가 25건 정도라, 임베딩이 각 학생의 우연한 점수 패턴까지 암기하기 시작하는 것이 원인이다. 드롭아웃만으로는 부족했고, 조기 종료를 적용한 B'가 epoch 6에서 멈추며 30.66%로 +2.3%p 개선되었다. 임베딩 행렬에 L2 규제를 거는 것도 추가 대안이다.

문제 2 — 고정 epoch 비교의 함정. 표만 보면 시그모이드(D, 30.02%)가 ReLU(C, 28.61%)보다 좋아 보인다. 그러나 그림 9의 곡선을 보면 ReLU는 1~2 epoch에 정점(약 30.9%)을 찍고 과적합으로 내려온 상태에서 측정된 것이고, 시그모이드는 늦게 출발한 탓에 10 epoch 시점이 마침 자신의 정점이었을 뿐이다. 즉 '10 epoch 후 정확도'라는 기준 자체가 모델마다 다른 시점을 비교하는 불공정한 잣대였다. 처음 결과를 보고 시그모이드가 더 좋은 활성화 함수라고 결론 내릴 뻔했는데, 곡선을 확인하고서야 측정 방법의 문제임을 알았다. 올바른 비교는 모든 모델에 조기 종료를 적용해 각자의 최적 시점에서 평가하는 것이다.

문제 3 — 정확도의 상한. 최선의 모델도 30.7%로, 이미지 분류처럼 90%대가 나오는 과제와 차원이 다르다. 원인은 데이터의 본질적 노이즈다: 같은 (학생 성향, 교수, 과목 조건) 조합이라도 실제 평점은 학생의 그날 기분, 성적, 보고서 채점 같은 기록되지 않은 요인에 좌우된다. 입력에 없는 정보는 어떤 모델도 맞힐 수 없으므로(베이즈 오차), 이 과제의 성과는 절대 정확도가 아니라 기준선 대비 개선(24.0% → 30.7%)과 오류의 질로 평가해야 한다. 실제로 혼동행렬(그림 11)을 보면 오류의 대부분이 인접 점수에 몰려 있고 ±1점 허용 정확도는 68.2%다 — 모델이 '대략적인 만족도'는 잡아내고 있다는 뜻이다.

문제 4 — 명목형 손실과 순서형 레이블의 불일치. 크로스 엔트로피는 점수의 순서를 모른다. 위 이론 절에서 설명한 기대 평점 방식으로 MAE를 1.013까지 줄였지만, 더 근본적으로는 출력을 1개 노드의 회귀로 바꾸거나 순서형 분류 손실을 쓰는 방법이 있다. 다만 회귀로 바꾸면 '확률 분포'라는 분류 출력의 장점을 잃으므로 절충이 필요하다.

문제 5 — 콜드 스타트. 임베딩은 학습에 등장한 ID에 대해서만 의미 있는 벡터를 가진다. 이번 분할에서는 학생 대부분이 학습셋에 한 번 이상 등장해 문제가 드러나지 않았지만, 완전히 새로운 학생/교수가 오면 그 임베딩은 학습되지 않은 무작위 초기값이다. 실제 시스템이라면 신규 ID는 전체 평균 임베딩으로 대체하거나 특성 기반 예측(모델 A)으로 후퇴하는 안전장치가 필요하다.

문제 6 — 실행 간 미세한 수치 변동. 난수 시드를 고정해도 TensorFlow의 CPU 병렬 연산 순서가 실행마다 달라 정확도가 $\pm 0.5\%$ 정도 변동했다. 보고서의 수치는 1회 실행 기준이며, 엄밀하게는 여러 번 실행한 평균과 표준편차를 보고하는 것이 바람직하다.

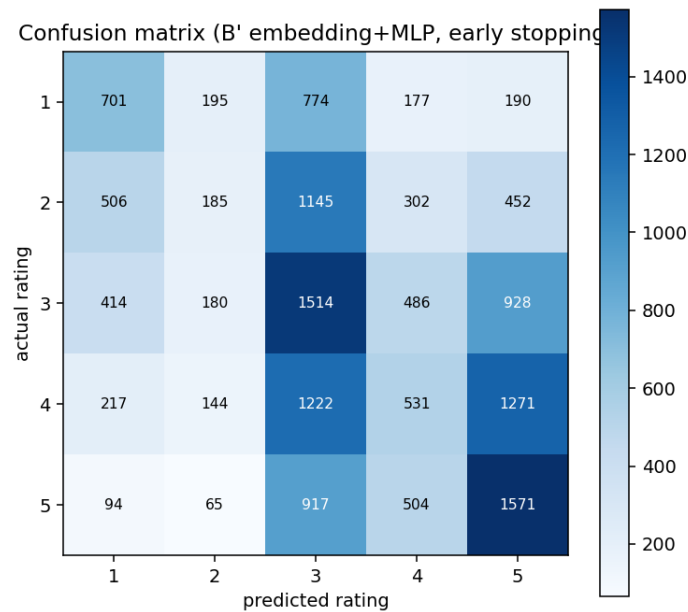


그림 11. 최종 모델(B')의 혼동행렬. 오류가 대각선 인접 칸에 집중되어 있다.

출처 및 참고자료

데이터셋

- Diamonds: ggplot2 (R) 내장 데이터셋, pydataset 패키지로 로드. <https://ggplot2.tidyverse.org/reference/diamonds.html>
- Wine: UCI Machine Learning Repository, scikit-learn load_wine으로 로드. <https://archive.ics.uci.edu/ml/datasets/wine>
- InstEval: ETH Zurich 강의평가 데이터, R lme4 패키지 수록, pydataset 패키지로 로드. Bates, Maechler, Bolker, Walker, lme4: Linear Mixed-Effects Models. <https://cran.r-project.org/package=lme4> (데이터 문서: InstEval)

라이브러리

- scikit-learn (<https://scikit-learn.org>), TensorFlow/Keras (<https://www.tensorflow.org>), NumPy, pandas, matplotlib, pydataset

강의자료

- 인공지능수학 week 11 (로지스틱 회귀, 2진 분류), week 12 (다중 클래스 분류), week 13 (딥러닝) 강의자료 및 보조 강의자료(로지스틱 회귀, 다중 클래스, Cross Entropy, 딥러닝, Keras/PyTorch)

생성형 AI 활용 상세 표기: Anthropic Claude를 다음 작업에 활용하였다 — (1) 세 프로젝트의 파이썬 코드 작성(주석 포함)과 디버깅, (2) 실험 실행 및 그림·수치 산출, (3) 본 보고서 초안의 구성과 문장 정리. 이론 내용은 수업 강의자료의 유도과 표기를 기준으로 작성하였으며, 최종 내용 확인과 제출 책임은 본인에게 있다.